

Lab 6

Question 3. The 0/1 Knapsack Algorithm typically uses an $n \times W$ table to store subproblem solutions. We can improve the space complexity by reducing the size of the table. Please refer to the above 0/1 Knapsack Algorithm and modify it to achieve a space complexity of $O(W)$.

We can reduce space from $O(n * W)$ to $O(W)$ by using a 1D array and iterating backward to avoid overwriting needed values:

```
int knapsackOptimized(int n, int W, int weights[], int values[]) {
    int dp[W + 1] = {0};

    for (int i = 0; i < n; ++i) {
        for (int w = W; w >= weights[i]; --w) {
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }

    return dp[W];
}
```

Question 4. We can solve the above problem using dynamic programming. What is the recursive transition function? Refer to the 0/1 knapsack recursive transition function and modify it accordingly.

Question 5. Please modify the 0/1 Knapsack Algorithm into an Unbounded Knapsack Algorithm with space complexity $O(W)$ and design an algorithm to reconstruct the combination of items with $O(W)$.

Optimized DP:

```
int unboundedKnapsack(int n, int W, int weights[], int values[]) {
    int dp[W + 1] = {0};

    for (int i = 0; i < n; ++i) {
        for (int w = weights[i]; w <= W; ++w) {
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }

    return dp[W];
}
```

Reconstruct Items:

```
vector<int> reconstructItems(int n, int W, int weights[], int values[]) {
    int dp[W + 1] = {0};
    int item[W + 1]; // To trace last item added
    fill(item, item + W + 1, -1);

    for (int i = 0; i < n; ++i) {
        for (int w = weights[i]; w <= W; ++w) {
            if (dp[w - weights[i]] + values[i] > dp[w]) {
                dp[w] = dp[w - weights[i]] + values[i];
                item[w] = i;
            }
        }
    }

    vector<int> counts(n, 0);
    int w = W;
    while (w > 0 && item[w] != -1) {
        int i = item[w];
        counts[i]++;
        w -= weights[i];
    }

    return counts; // counts[i] = number of times item i was used
}
```

Question 6. Please refer to the above Bounded Knapsack Algorithm and modify it using binary optimization to achieve a time complexity of $O(n \cdot W \cdot \log c_{\max})$, where $c_{\max}$ denotes the maximum copy limit among all items.

```
int boundedKnapsackOptimized(int n, int W, int weights[], int values[], int counts[]) {
    vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; ++i) {
        int w = weights[i], v = values[i], c = counts[i];
        for (int k = 1; c > 0; k <= 1) {
            int actualCount = min(k, c);
            int weight_k = actualCount * w;
            int value_k = actualCount * v;

            for (int j = W; j >= weight_k; --j) {
                dp[j] = max(dp[j], dp[j - weight_k] + value_k);
            }

            c -= actualCount;
        }
    }

    return dp[W];
}
```

Lab 7

Question 2 .In basketball, each successful shot gains either 2 or 3 points. Please write a program to calculate how many distinct ways that a team can get exactly \$n\$ points.

```
#include <iostream>
#include <vector>

int waysToScore(int n, std::vector<int>& memo) {
    if (n == 0) return 1;
    if (n < 0) return 0;
    if (memo[n] != -1) return memo[n];

    memo[n] = waysToScore(n - 2, memo) + waysToScore(n - 3, memo);
    return memo[n];
}

int main() {
    int n;
    std::cout << "Enter the total points: ";
    std::cin >> n;

    std::vector<int> memo(n + 1, -1);
    int result = waysToScore(n, memo);
    std::cout << "Number of ways to get exactly " << n << " points: " << result << std::endl;

    return 0;
}
```

Question 3 Some of the stairs were broken. We can not step on the broken stairs. Given the broken stairs by $B = \{b_1, b_2, \dots, b_m\}$, $1 \leq b_1 < b_2 < \dots < b_m \leq n$, how many ways can we climb to total n stairs?

```
#include <iostream>
#include <vector>
#include <unordered_set>

int climbStairsWithBroken(const std::vector<int>& broken, int n) {
    std::unordered_set<int> brokenSet(broken.begin(), broken.end());
    std::vector<int> dp(n + 1, 0);
    dp[0] = 1; // 1 way to stand before starting

    for (int i = 1; i <= n; ++i) {
        if (brokenSet.count(i)) {
            dp[i] = 0; // broken step
        } else {
            int waysFrom1 = (i - 1 >= 0) ? dp[i - 1] : 0;
            int waysFrom2 = (i - 2 >= 0) ? dp[i - 2] : 0;
            dp[i] = waysFrom1 + waysFrom2;
        }
    }
    return dp[n];
}

int main() {
    int n, m;
    std::cout << "Enter total steps: ";
    std::cin >> n;
    std::cout << "Enter number of broken steps: ";
    std::cin >> m;

    std::vector<int> broken(m);
    std::cout << "Enter broken steps: ";
    for (int i = 0; i < m; ++i)
        std::cin >> broken[i];

    int ways = climbStairsWithBroken(broken, n);
    std::cout << "Number of ways to climb " << n << " stairs with broken steps: " << ways
    << std::endl;

    return 0;
}
```

}

Question 4 Will sorting C by increasing order improve the performance or not? Why?

Question 5 Since $f(C_k, *)$ only depends on $f(C_k, *)$ and $f(C_{k-1}, *)$, we can cache the data in $2 \times m$. The skill named **rolling**. Please rewrite above algorithm to a rolling table version.

> Hint: store $f(C_k, m)$ to `dp[k%2][m]`

// C++ style pseudocode

```
int n = number_of_coins;
int M = target_amount;
vector<int> coins = C; // coin denominations
```

```
// dp array with 2 rows and M+1 columns
// dp[0][m] and dp[1][m] will be used alternatively
vector<vector<int>> dp(2, vector<int>(M + 1, INF));
```

```
// Initialization: dp for 0 coins
```

```
for (int m = 0; m <= M; ++m) {  
    dp[0][m] = (m == 0) ? 0 : INF; // 0 coins needed to make 0 amount  
}
```

```
for (int k = 1; k <= n; ++k) {  
    int curr = k % 2;  
    int prev = (k - 1) % 2;
```

```
    dp[curr][0] = 0; // 0 coins to make amount 0
```

```
    for (int m = 1; m <= M; ++m) {  
        if (coins[k - 1] <= m) {  
            // Either don't use coin k, or use it and add one coin  
            dp[curr][m] = min(dp[prev][m], 1 + dp[curr][m - coins[k - 1]]);  
        } else {  
            dp[curr][m] = dp[prev][m];  
        }  
    }  
}
```

```
// Answer is in dp[n % 2][M]  
int answer = dp[n % 2][M];
```

Lab 8

Problem 1.1

Problem 1.2

Using tabulation method

```
#include <iostream>
#include <vector>
using namespace std;

// ##### STUDENT'S IMPLEMENTATION AREA START #####
long long count_ways_to_climb(int n) {
    if (n < 0) return 0;
    std::vector<long long> dp(n + 1, 0);
    dp[0] = 1; // 1 way to stay at the ground (doing nothing)

    for (int i = 1; i <= n; ++i) {
        if (i >= 1) dp[i] += dp[i - 1];
        if (i >= 2) dp[i] += dp[i - 2];
        if (i >= 3) dp[i] += dp[i - 3];
    }

    return dp[n];
}
// ##### STUDENT'S IMPLEMENTATION AREA END #####

int main () {

    cout << "--- Fibonacci / Climbing Stairs ---" << endl;

    cout << "\n--- Climbing Stairs (1, 2, or 3 steps) ---" << endl;
    cout << "Ways to climb 0 stairs: " << count_ways_to_climb(0) << endl; // Expected: 1
    cout << "Ways to climb 1 stair: " << count_ways_to_climb(1) << endl; // Expected: 1
    cout << "Ways to climb 2 stairs: " << count_ways_to_climb(2) << endl; // Expected: 2
    cout << "Ways to climb 3 stairs: " << count_ways_to_climb(3) << endl; // Expected: 4
    cout << "Ways to climb 4 stairs: " << count_ways_to_climb(4) << endl; // Expected: 7
    cout << "Ways to climb 5 stairs: " << count_ways_to_climb(5) << endl; // Expected: 13

    return 0;
}
```

Problem 2.1

Problem 2.2

```
#include <iostream>
#include <vector>
#include <algorithm> // For std::max and std::max_element
using namespace std;

// ##### STUDENT'S IMPLEMENTATION AREA START #####
// ##### 學生的實作區域 START #####
int longest_increasing_subsequence(const std::vector<int>& nums) {
    // 1. Handle empty input
    if (nums.empty()) return 0;

    // 2. Get the size
    int n = nums.size();

    // 3. Create a dp vector and initialize to 1
    std::vector<int> dp(n, 1);

    // 4. Fill the dp vector using the recurrence relation
    for (int i = 1; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (nums[j] < nums[i]) {
                dp[i] = std::max(dp[i], dp[j] + 1);
            }
        }
    }

    // 5. Return the max value in dp
    return *std::max_element(dp.begin(), dp.end());
}
// ##### STUDENT'S IMPLEMENTATION AREA END #####
// ##### 學生的實作區域 END #####

int main() {

    cout << "\n--- Longest Increasing Subsequence ---" << endl;

    vector<int> nums1 = {10, 9, 2, 5, 3, 7, 101, 18};
    cout << "LIS length for {10, 9, 2, 5, 3, 7, 101, 18}: "
        << longest_increasing_subsequence(nums1) << endl; // Expected: 4

    vector<int> nums2 = {0, 1, 0, 3, 2, 3};
    cout << "LIS length for {0, 1, 0, 3, 2, 3}: "
        << longest_increasing_subsequence(nums2) << endl; // Expected: 4
```

```
vector<int> nums3 = {7, 7, 7, 7, 7};
cout << "LIS length for {7, 7, 7, 7, 7}: "
    << longest_increasing_subsequence(nums3) << endl; // Expected: 1

vector<int> nums4 = {3, 10, 2, 1, 20};
cout << "LIS length for {3, 10, 2, 1, 20}: "
    << longest_increasing_subsequence(nums4) << endl; // Expected: 3

vector<int> nums5 = {};
cout << "LIS length for {}: "
    << longest_increasing_subsequence(nums5) << endl; // Expected: 0

return 0;
}
```

Problem 1.2 using memoization

c++

```
#include <iostream>
#include <map>

// ##### STUDENT'S IMPLEMENTATION AREA START #####
std::map<int, long long> memo; // memoization table

long long count_ways_to_climb(int n) {
    if (n < 0) return 0;    // No way to climb negative stairs
    if (n == 0) return 1;   // One way to stay at the ground (do nothing)

    if (memo.find(n) != memo.end()) return memo[n]; // return cached result

    // Store the result in memo after recursive calls
    memo[n] = count_ways_to_climb(n - 1) +
              count_ways_to_climb(n - 2) +
              count_ways_to_climb(n - 3);

    return memo[n];
}
// ##### STUDENT'S IMPLEMENTATION AREA END #####

int main_fib_climb() {
    using namespace std;
    cout << "--- Fibonacci / Climbing Stairs (Memoization) ---" << endl;

    // Clear memo before testing, if reused
    memo.clear();
    cout << "Ways to climb 0 stairs: " << count_ways_to_climb(0) << endl; // Expected: 1
    cout << "Ways to climb 1 stair: " << count_ways_to_climb(1) << endl; // Expected: 1
    cout << "Ways to climb 2 stairs: " << count_ways_to_climb(2) << endl; // Expected: 2
    cout << "Ways to climb 3 stairs: " << count_ways_to_climb(3) << endl; // Expected: 4
    cout << "Ways to climb 4 stairs: " << count_ways_to_climb(4) << endl; // Expected: 7
    cout << "Ways to climb 5 stairs: " << count_ways_to_climb(5) << endl; // Expected: 13

    return 0;
}
```